



PUCIT

Data Science Assignment 1 (BSEF22)

Submission to: Dr. Mufassra Naz

Submission by: BSEF22M517 Fiza Haider
BSEF22M526 Noor Fatima

Submission date: 1st, January 2026

Table of Contents

1.	Initial Data Exploration.....	3
2.	Problem Definition.....	3
3.	Data Splitting	3
4.	Model Selection.....	7
5.	Model Training.....	7
6.	Model Evaluation	9
7.	Model Comparison.....	12
8.	Feature Importance / Interpretation.....	14
9.	Conclusion.....	16

Data Science Assignment 2 (BSEF22)

1. Initial Data Exploration

Group Members:

- BSEF22M517 Fiza Haider
- BSEF22M526 Noor Fatima

Dataset Information

Dataset Name: NYPD Hate Crimes

File Format: CSV

Dataset Description (2–3 lines):

This dataset contains records of hate crimes reported to NYPD, including features such as year, month, precinct, borough, county, offense type, bias motive description and more.

Source Link (URL):

<https://catalog.data.gov/dataset/nypd-hate-crimes/resource/6a431837-5576-420d-8857-e1beee49de2d>

Reason for Choosing this Dataset:

- Real-world dataset suitable for classification tasks.
- Contains categorical and numerical features.
- Enables analysis of anti-Jewish vs other biases.

2. Problem Definition

- **Task:** Classification – Predict Anti-Jewish vs Other Bias Crimes
- **Input Features (X):** Complaint Year, Month, Precinct Code, Borough, County, Law Code, Offense Description, PD Code, Offense Category
- **Target (y):** target (1 = Anti-Jewish, 0 = Other)
- **Goal:** Identify patterns in historical crime data to predict Anti-Jewish incidents

3. Data Splitting

3.1 Libraries Loading

```
import pandas as pd
```

```
import numpy as np
```

```

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, classification_report, roc_auc_score, roc_curve

import warnings

warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-darkgrid')

sns.set_palette("husl")

```

3.2 Dataset Loading & Initial Insights

```

df = pd.read_csv('NYPD_Hate_Crimes.csv')

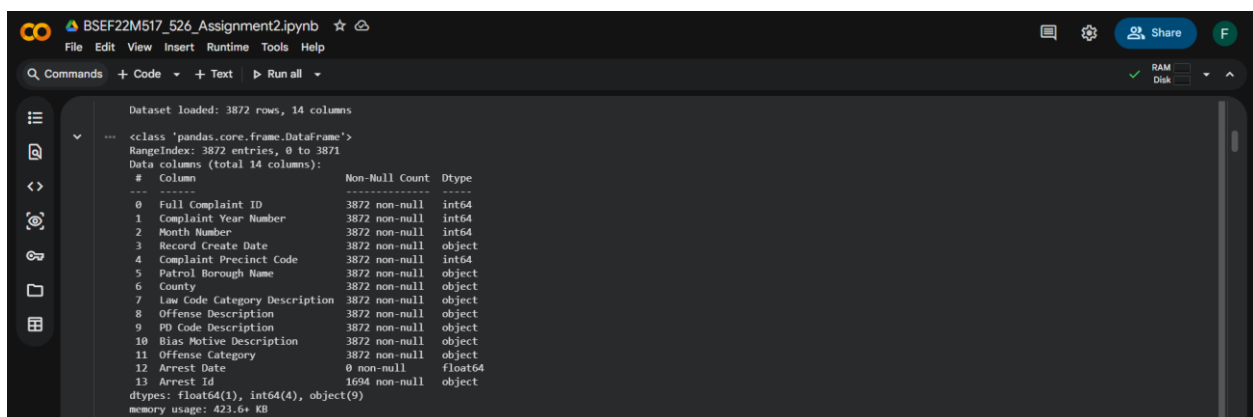
print(df.info())

df.head()

print(df.columns.tolist())

```

Info: 3872 rows, 14 columns including Complaint ID, Year, Month, Precinct, Borough, County, Law Code, Offense Description, Bias Motive, etc.



```

Dataset loaded: 3872 rows, 14 columns

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3872 entries, 0 to 3871
Data columns (total 14 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Full Complaint ID                    3872 non-null  int64
 1   Complaint Year Number                3872 non-null  int64
 2   Month Number                        3872 non-null  int64
 3   Record Create Date                  3872 non-null  object
 4   Complaint Precinct Code              3872 non-null  int64
 5   Patrol Borough Name                 3872 non-null  object
 6   County                              3872 non-null  object
 7   Law Code Category Description        3872 non-null  object
 8   Offense Description                 3872 non-null  object
 9   PD Code Description                 3872 non-null  object
10   Bias Motive Description              3872 non-null  object
11   Offense Category                    3872 non-null  object
12   Arrest Date                         0 non-null     float64
13   Arrest Id                           1694 non-null  object
dtypes: float64(1), int64(4), object(9)
memory usage: 423.6+ KB
None

```

	Full Complaint ID	Complaint Year Number	Month Number	Record Create Date	
0	201906012119317	2019	1	01/23/2019	
1	201906012175717	2019	2	02/25/2019	
2	201906012180117	2019	2	02/27/2019	
3	201906012273417	2019	4	04/16/2019	
4	201906012413717	2019	6	06/20/2019	

	Complaint Precinct Code	Patrol Borough Name	County	
0	60	PATROL BORO BKLYN SOUTH	KINGS	
1	60	PATROL BORO BKLYN SOUTH	KINGS	
2	60	PATROL BORO BKLYN SOUTH	KINGS	
3	60	PATROL BORO BKLYN SOUTH	KINGS	
4	60	PATROL BORO BKLYN SOUTH	KINGS	

	Law Code Category	Description	Offense Description	
0	FELONY	MISCELLANEOUS	PENAL LAW	
1	FELONY	MISCELLANEOUS	PENAL LAW	
2	FELONY	MISCELLANEOUS	PENAL LAW	
3	FELONY	MISCELLANEOUS	PENAL LAW	
4	FELONY	MISCELLANEOUS	PENAL LAW	

	PD Code	Description	Bias Motive	Description	
0	AGGRAVATED HARASSMENT	1	ANTI-JEWISH		
1	AGGRAVATED HARASSMENT	1	ANTI-JEWISH		
2	AGGRAVATED HARASSMENT	1	ANTI-JEWISH		
3	AGGRAVATED HARASSMENT	1	ANTI-JEWISH		
4	AGGRAVATED HARASSMENT	1	ANTI-JEWISH		

	Offense Category	Arrest Date	Arrest Id
0	Religion/Religious Practice	NaN	NaN
1	Religion/Religious Practice	NaN	NaN
2	Religion/Religious Practice	NaN	NaN
3	Religion/Religious Practice	NaN	NaN
4	Religion/Religious Practice	NaN	NaN

Column Names:

['Full Complaint ID', 'Complaint Year Number', 'Month Number', 'Record Create Date', 'Complaint Precinct Code', 'Patrol Borough Name', 'County', 'Law Code Category Description', 'Of

3.3 Target Variable Creation:

Binary target: Anti-Jewish = 1, Other = 0

```
df['target'] = (df['Bias Motive Description'] == 'ANTI-JEWISH').astype(int)
```

Distribution check

```
target_dist = df['target'].value_counts()
```

```
print("Anti-Jewish (1):", target_dist.get(1, 0), f"({target_dist.get(1, 0)/len(df)*100:.1f}%")
```

```
print("Other Biases (0):", target_dist.get(0, 0), f"({target_dist.get(0, 0)/len(df)*100:.1f}%")
```

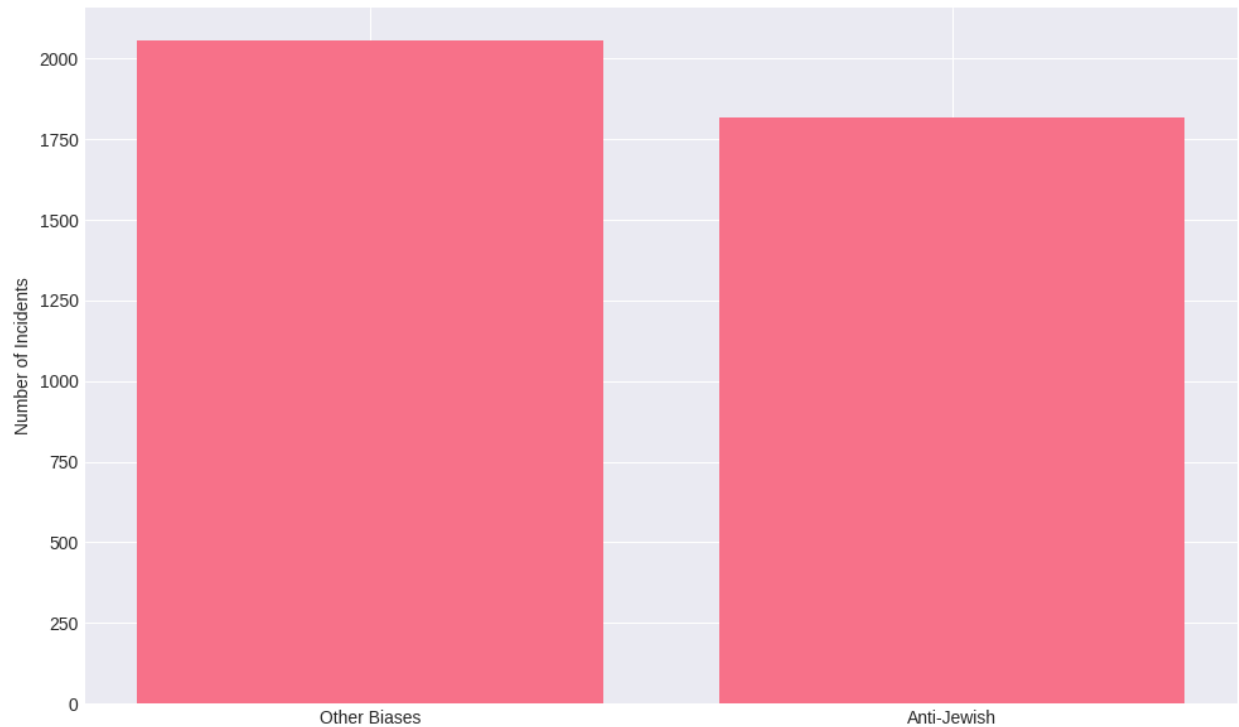
```
plt.figure(figsize=(10, 6))
```

```
plt.bar(['Other Biases', 'Anti-Jewish'],[target_dist.get(0, 0), target_dist.get(1, 0)])
```

```
plt.ylabel('Number of Incidents')
```

```
plt.tight_layout()
```

```
plt.show()
```



3.3 Handling Missing Values

for col in df.columns:

if df[col].dtype == 'object':

df[col] = df[col].fillna('Unknown')

else:

df[col] = df[col].fillna(df[col].median())

Explanation: Ensures no missing values exist before processing.

3.4 Train-Test Split

features = [

'Complaint Year Number',

'Month Number',

'Complaint Precinct Code',

'Patrol Borough Name',

'County',

'Law Code Category Description',

```
'Offense Description',  
'PD Code Description',  
'Offense Category'  
]
```

```
X = df[features].copy()  
y = (df['Bias Motive Description'] == 'ANTI-JEWISH').astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

Note: 80–20 split ensures enough training data while keeping test set representative.



4. Model Selection

- **Logistic Regression:** Interpretable baseline
- **Decision Tree:** Handles categorical features
- **Random Forest:** Robust ensemble classifier

Reason: Effective for predicting Anti-Jewish vs Other crimes.

5. Model Training

5.1 Encoding Categorical Features

```
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()  
label_encoders = {}  
for col in categorical_cols:  
    le = LabelEncoder()
```

```
X[col] = le.fit_transform(X[col].astype(str))
```

```
label_encoders[col] = le
```

5.2 Feature Scaling

```
numeric_cols = X_train.select_dtypes(include=np.number).columns.tolist()
```

```
scaler = StandardScaler()
```

```
X_train_scaled = X_train.copy()
```

```
X_test_scaled = X_test.copy()
```

```
X_train_scaled[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
```

```
X_test_scaled[numeric_cols] = scaler.transform(X_test[numeric_cols])
```

5.3 Training Models

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
models = {
```

```
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
```

```
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
```

```
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
```

```
}
```

```
for name, model in models.items():
```

```
    if name == 'Logistic Regression':
```

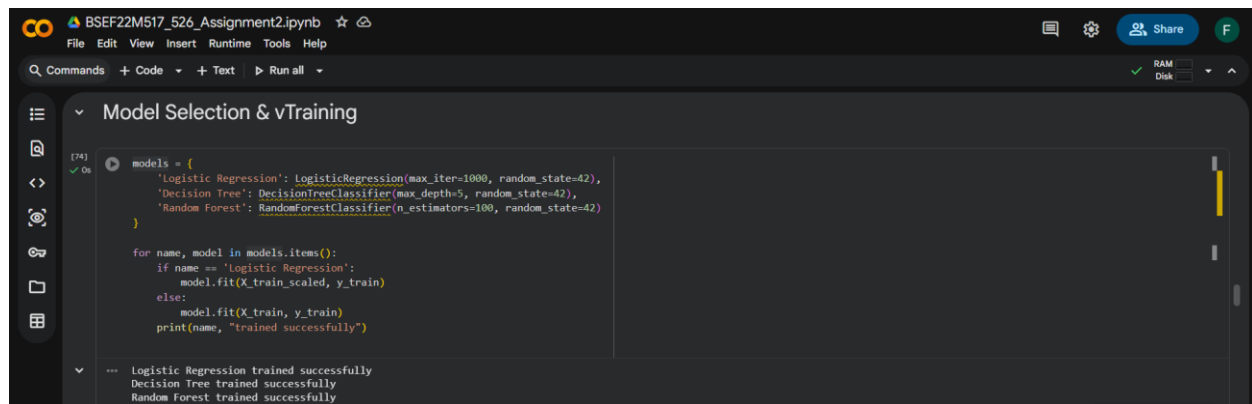
```
        model.fit(X_train_scaled, y_train)
```

```
    else:
```

```
        model.fit(X_train, y_train)
```

```
    print(name, "trained successfully")
```

All models trained successfully on the training dataset.



The screenshot shows a Jupyter Notebook titled "BSEF22M517_526_Assignment2.ipynb". The code is in a cell labeled "[74]:". The code defines a dictionary of models and a loop to train them. The output shows that Logistic Regression, Decision Tree, and Random Forest were trained successfully.

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

for name, model in models.items():
    if name == 'Logistic Regression':
        model.fit(X_train_scaled, y_train)
    else:
        model.fit(X_train, y_train)
    print(name, "trained successfully")
```

Logistic Regression trained successfully
Decision Tree trained successfully
Random Forest trained successfully

6. Model Evaluation

```
results = {}
```

```
roc_data = {}
```

```
for name, model in models.items():
```

```
    if name == 'Logistic Regression':
```

```
        y_pred = model.predict(X_test_scaled)
```

```
        y_pred_proba = model.predict_proba(X_test_scaled)[:,-1]
```

```
    else:
```

```
        y_pred = model.predict(X_test)
```

```
        y_pred_proba = model.predict_proba(X_test)[:,-1]
```

```
        accuracy = accuracy_score(y_test, y_pred)
```

```
        precision = precision_score(y_test, y_pred)
```

```
        recall = recall_score(y_test, y_pred)
```

```
        f1 = f1_score(y_test, y_pred)
```

```
        auc = roc_auc_score(y_test, y_pred_proba)
```

```
    results[name] = {'Accuracy': accuracy, 'Precision': precision,
```

```
                    'Recall': recall, 'F1-Score': f1, 'AUC-ROC': auc}
```

```

fpr, tpr, _ = roc_curve(y_test, y_pred_proba)

roc_data[name] = (fpr, tpr, auc)

print(f"\n{name} Metrics:")

print("Accuracy:", round(accuracy,3))

print("Precision:", round(precision,3))

print("Recall:", round(recall,3))

print("F1-Score:", round(f1,3))

print("AUC-ROC:", round(auc,3))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

Explanation: Evaluate each model using common classification metrics.

```

Logistic Regression Metrics:
Accuracy: 0.694
Precision: 0.687
Recall: 0.64
F1-Score: 0.663
AUC-ROC: 0.751
Confusion Matrix:
[[305 106]
 [131 233]]

Decision Tree Metrics:
Accuracy: 0.941
Precision: 0.892
Recall: 0.995
F1-Score: 0.94
AUC-ROC: 0.973
Confusion Matrix:
[[367  44]
 [  2 362]]

Random Forest Metrics:
Accuracy: 0.948
Precision: 0.926
Recall: 0.967
F1-Score: 0.946
AUC-ROC: 0.978
Confusion Matrix:
[[383  28]
 [ 12 352]]

```

Confusion Matrix Analysis

Pasted from Google Collab Output

1. Logistic Regression

Confusion Matrix:

$$\begin{bmatrix} 305 & 106 \\ 131 & 233 \end{bmatrix}$$

- **True Negatives (TN) = 305** -> correctly predicted "Other Bias" crimes

- **False Positives (FP) = 106** -> "Other Bias" crimes incorrectly predicted as Anti-Jewish
- **False Negatives (FN) = 131** -> Anti-Jewish crimes that were missed
- **True Positives (TP) = 233** -> correctly predicted Anti-Jewish crimes

Insight:

- Logistic Regression missed 131 Anti-Jewish incidents, resulting in a **moderate recall of 0.64**.
- **Precision is 0.687**, meaning some "Other Bias" crimes were misclassified as Anti-Jewish.
- While the model is **highly interpretable**, it is **less effective at detecting the minority class**.

2. Decision Tree

Confusion Matrix:

$$\begin{bmatrix} 367 & 44 \\ 2 & 362 \end{bmatrix}$$

- **TN = 367, FP = 44, FN = 2, TP = 362**

Insight:

- The model missed only **2 Anti-Jewish incidents**, giving an **extremely high recall of 0.995**.
- Precision is slightly lower than Random Forest (**0.892**) due to some false positives.
- Decision Tree is **very effective at capturing Anti-Jewish crimes**, though it may **slightly overfit**.

3. Random Forest

Confusion Matrix:

$$\begin{bmatrix} 383 & 28 \\ 12 & 352 \end{bmatrix}$$

- **TN = 383, FP = 28, FN = 12, TP = 352**

Insight:

- High recall (**0.967**) ensures most Anti-Jewish incidents are correctly identified.
- High precision (**0.926**) means fewer false positives than the Decision Tree.
- Random Forest achieves the **best overall balance**, reflected in the **highest F1-score (0.946) and AUC-ROC (0.978)**.

- This makes it the **most reliable model for detecting minority-class incidents**.

7. Model Comparison

```
results_df = pd.DataFrame(results).T
```

```
results_df.round(3)
```

Observation: Random Forest showed the highest F1-Score and recall.

Pasted from Google Collab Output

	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	0.694	0.687	0.640	0.663	0.751
Decision Tree	0.941	0.892	0.995	0.940	0.973
Random Forest	0.948	0.926	0.967	0.946	0.978

```
# Bar plots for metrics
```

```
fig, axes = plt.subplots(2,3, figsize=(18,12))
```

```
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC-ROC']
```

```
for i, metric in enumerate(metrics):
```

```
    ax = axes[i//3, i%3]
```

```
    results_df[metric].plot(kind='bar', ax=ax)
```

```
    ax.set_title(metric + " Comparison")
```

```
    ax.set_ylim(0,1)
```

```
    ax.grid(alpha=0.3)
```

```
# ROC Curve
```

```
ax = axes[1,2]
```

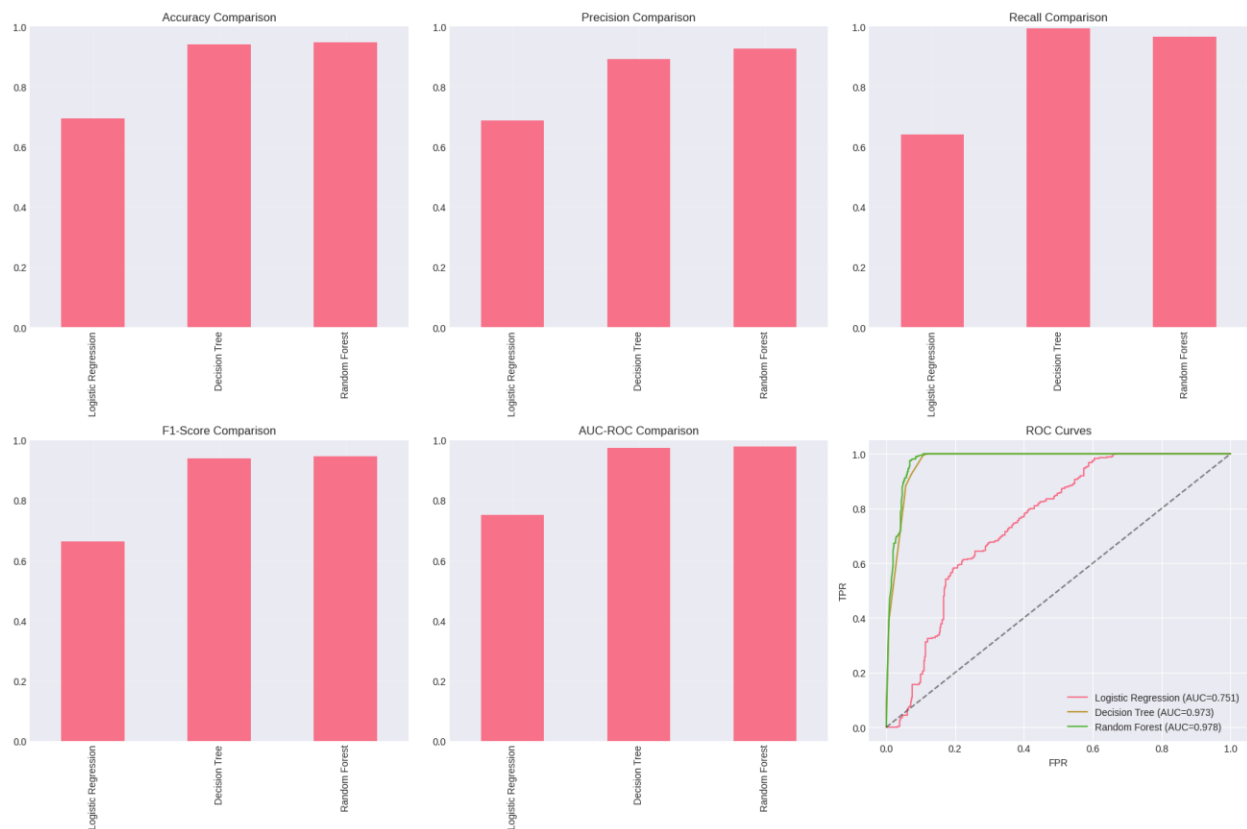
```
for name, (fpr, tpr, auc) in roc_data.items():
```

```
    ax.plot(fpr, tpr, label=f"{name} (AUC={auc:.3f})")
```

```

ax.plot([0,1],[0,1],'k--', alpha=0.5)
ax.set_title("ROC Curves")
ax.set_xlabel("FPR")
ax.set_ylabel("TPR")
ax.legend()
plt.tight_layout()
plt.show()

```



Insights:

Logistic Regression

- Achieved moderate performance with **Accuracy = 0.694**, **Recall = 0.64**, and **F1-Score = 0.663**.
- Missed a significant number of Anti-Jewish incidents (**131 FN**), showing it struggles to detect the minority class.

- Precision = 0.687 indicates some “Other Bias” crimes were incorrectly flagged as Anti-Jewish.
- Strength: highly interpretable, Weakness: lower recall and F1-Score make it less reliable for minority-class prediction.

Decision Tree

- High recall (**0.995**) and F1-Score (**0.940**) demonstrate excellent ability to detect Anti-Jewish incidents, missing only 2 cases.
- Precision = 0.892 indicates some false positives exist.
- Strength: effective for minority-class detection and handles categorical features well.
- Weakness: slightly overfits due to single-tree nature, variance can affect robustness.

Random Forest

- Best overall performance: **Accuracy = 0.948, Recall = 0.967, F1-Score = 0.946, AUC-ROC = 0.978.**
- Missed only 12 Anti-Jewish incidents and produced fewer false positives than the Decision Tree.
- Strength: balances sensitivity (recall) and precision, making it highly reliable for minority-class prediction.
- Robust ensemble approach reduces overfitting and variance, outperforming single-tree models.

8. Feature Importance / Interpretation

8.1 Random Forest Feature Importance

```
rf_model = models['Random Forest']

feature_importance = pd.DataFrame({
    'Feature': features,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

feature_importance
```

	Feature	Importance
8	Offense Category	0.554285
7	PD Code Description	0.136505

	Feature	Importance
2	Complaint Precinct Code	0.084968
6	Offense Description	0.058885
1	Month Number	0.058626
0	Complaint Year Number	0.044193
3	Patrol Borough Name	0.030828
5	Law Code Category Description	0.016888
4	County	0.014823

Pasted from Google Collab Output

8.2 Logistic Regression Coefficients

```
lr_model = models['Logistic Regression']
coeff_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': lr_model.coef_[0]
}).sort_values('Coefficient', ascending=False)
coeff_df.head(5)
```

Explanation: Shows which features influence predictions most.

	Feature	Coefficient
8	Offense Category	0.811005
6	Offense Description	0.331588
4	County	0.248479
2	Complaint Precinct Code	0.079604
0	Complaint Year Number	0.032109

Pasted from Google Collab Output

9. Conclusion

- **Best Model:** Random Forest achieves the highest F1-Score and recall while maintaining strong precision and AUC-ROC. It reliably identifies Anti-Jewish incidents with minimal false negatives and false positives.

Key Insights:

- Minority-class detection (Anti-Jewish incidents) benefits from ensemble methods like Random Forest.
- Certain features (Offense Category, PD Code Description, Complaint Precinct Code) are highly predictive.

Limitations:

- Dataset imbalance may bias models toward majority class.
- Simplified categorical encoding (LabelEncoder) could limit capturing feature relationships.
- The current analysis does not optimize hyperparameters extensively.

Future Improvements:

- Use **SMOTE or class weighting** to handle minority-class imbalance more effectively.
- Explore advanced **feature engineering**, including text features from offense descriptions.
- Experiment with additional models for comparison.